

Estado de Tareas en Memoria

Imaginá que tenemos una instancia de proceso `inst:101`, un usuario `u:20`, y el rol `soporte`.

- **Paso 1: Guardar el estado de la instancia y las tareas pendientes** Guardamos la instancia como un `Hash` y asignamos tareas pendientes al rol usando un `Set` (o `List`):

```
code
# 1. Guardar estado general de la instancia con TTL de 1 hora
(3600 seg)
HSET instancia:101 estado "EN_PROCESO" tarea_actual "t_500"
asignado "u:20" EXPIRE instancia:101 3600

# 2. Agregar tareas pendientes a la lista del rol soporte
SADD tareas_pendientes:rol:soporte "t_500" "t_501" "t_502"
```

- **Paso 2: Consultar la información rápidamente ($O(1)$)**

```
code
# Leer el estado completo de la instancia
HGETALL instancia:101

# Ver todas las tareas pendientes del rol soporte
SMEMBERS tareas_pendientes:rol:soporte
```

- **Paso 3: Completar la tarea y actualizar/invalidar** Cuando el usuario `u:20` completa la tarea `t_500`, quitamos la tarea del set e invalidamos o actualizamos el hash:

```
code
# Remover la tarea completada del grupo de pendientes
```

```
SREM tareas_pendientes:rol:soporte "t_500"
```

```
# Opción A: Invalidar (borrar para forzar recarga limpia desde DB)
```

```
DEL instancia:101
```

```
# Opción B: Actualizar estado en caliente
```

```
HSET instancia:101 estado "COMPLETADA" tarea_actual ""
```

Ahora que viste cómo interactúan `HSET`, `SADD`, `SREM`, `EXPIRE` y `DEL`, vamos a resolver paso a paso el ejercicio de parcial de **FlowOps**:

2.A — Diagnóstico

- **Tres causas posibles de estados desactualizados:**

1. *Falta de invalidación/actualización*: La aplicación actualizó la base de datos persistente (SQL/Cassandra) pero no ejecutó el `DEL` o `HSET` en Redis.
2. *TTL excesivamente largo*: El dato mutó en la fuente principal, pero Redis sigue sirviendo el valor en memoria porque la clave aún no expiró.
3. *Condición de carrera (Race Condition)*: Una lectura concurrente que sufrió un *Cache Miss* leyó un dato viejo de la base principal y sobrescribió en Redis una actualización más reciente.

- **Diferencia entre actualizar e invalidar caché:**

- *Actualizar (`SET` / `HSET`)*: Escribe el nuevo valor directamente en Redis. Mantiene latencias bajas en lecturas posteriores, pero consume CPU/red en escrituras y puede generar inconsistencias si la escritura falla a medias.
- *Invalidar (`DEL`)*: Elimina la clave de Redis. La siguiente lectura

- **invalidar (DEL)**: Elimina la clave de Redis. La siguiente lectura sufrirá un *Cache Miss* y leerá la verdad absoluta de la base de datos persistente para repoblar la caché (*Cache-Aside*). Es más seguro y simple.

- **Riesgo si el TTL es demasiado largo**: Los usuarios leen datos viejos/obsoletos (*stale data*) durante mucho tiempo si falla la invalidación manual.
- **Riesgo si el TTL es demasiado corto**: *Cache Stampede* / *Thundering Herd* (muchas consultas concurrentes van directo a la base de datos persistente saturándola al expirar la clave) y aumento innecesario de lecturas a disco.
- **Datos que pueden reconstruirse si Redis se pierde**: Todos los datos derivados de la lógica de negocio persistente (estado de instancias, historial de tareas, perfiles). Lo único que **no** se reconstruye de forma idéntica son contadores volátiles efímeros en tiempo real (ej. rate limits en la ventana actual) a menos que se hayan sincronizado previamente.

2.B — Diseño de claves y estructuras

Necesidad	Clave propuesta	Tipo de dato Redis	TTL
Estado actual de instancia	<code>instancia: {id_instancia}:estado</code>	Hash (o String con JSON)	Corto/ Medio (ej. <code>300s</code> / 5 min)
Tareas pendientes	<code>tareas:rol: {id_rol}:pendientes</code>	Set (o Sorted Set si	Medio (ej. <code>600s</code> /

por rol		tienen prioridad)	10 min)
Sesión de usuario	<code>sesion:token:{token_id}</code>	Hash o String	Duración de sesión (ej. <code>86400s</code> / 24h)
Rate limit por tenant	<code>ratelimit:tenant:{id_tenant}:{minuto_epoch}</code>	String (entero con <code>INCR</code>)	Muy corto (ej. <code>60s</code>)
Contador de consultas de estado	<code>metricas:consultas:{id_tenant}</code>	String (<code>INCRBY</code>) o Hash	Sin TTL o largo (ej. <code>86400s</code>)

2.C — Análisis de consistencia operativa (Completar Tarea)

Secuencia recomendada:

- Garantizar idempotencia:** La aplicación verifica en la base de datos o mediante un lock (`SET lock:tarea:{id_tarea} "1" NX EX 5`) que la tarea no fue completada previamente.
- Actualizar la persistencia primero:** Se ejecuta la transacción en la base de datos principal (Postgres/Cassandra) cambiando el estado a `COMPLETADA` y asentando el *timestamp*.
- Invalidar / Actualizar en Redis:**
 - Se elimina la tarea del set de pendientes: `SREM tareas:rol:{id_rol}:pendientes "{id_tarea}"`

- Se invalida la clave de la instancia para evitar inconsistencias:

```
DEL instancia:{id_instancia}:estado
```

- Se incrementa el contador de operaciones: INCR

```
metricas:tareas_completadas:{id_tenant}
```

4. **Respuesta al usuario:** Se retorna HTTP 200 OK informando éxito en la operación.

Manejo de fallos y detalles de consistencia:

- **Qué pasa si Redis no responde:** La transacción en la base de datos persistente ya fue confirmada (el dato real está a salvo). Se registra un warning en el log del sistema y la aplicación sigue operando; las siguientes lecturas irán temporalmente a la base principal hasta que Redis se recupere.
- **Qué respuesta recibe el usuario:** Éxito (200 OK), ya que el cambio persistente se logró.
- **Cómo se evita duplicar la operación:** Mediante clave de idempotencia o chequeo condicional en la base de datos relacional (UPDATE ... WHERE estado = 'PENDIENTE').

Cuatro comandos Redis conceptuales utilizados en el flujo:

1. SET lock:tarea:500 "token" NX EX 5 *(Bloqueo distribuido para evitar doble ejecución)*
2. SREM tareas:rol:soporte "500" *(Quitar la tarea de la lista de pendientes)*
3. DEL instancia:101:estado *(Invalidar la caché para forzar lectura fresca)*
4. INCR metricas:completadas:tenant_A *(Contador atómico de métricas)*